



NoSQL data warehouse optimizing models: A comparative study of column-oriented approaches

Mohamed Mouhiha^{id,*}, Abdelfettah Mabrouk

Chouaib Doukkali University, ESTSB, ELITES Lab, El Jadida, Morocco

ARTICLE INFO

Keywords:

Big data
Big data warehouse
Column-oriented NoSQL database
Optimization

ABSTRACT

There is a great challenge when building an efficient Big Data Warehouse (DW) from the traditional data warehouse which used to handle the large datasets. Several presented solutions concentrate on the conversion of a standard DW to an columnar model, especially for direct and traditional data sources. Though there have been many successful algorithms that apply data clustering methods, these approaches also come with their fair share of limitations. This paper provides a comprehensive review of the existing methods, both tuned and out-of-the box, exposing their strengths and weaknesses. Further, a comparative study of the different options is always conducted to compare and assess them.

1. Introduction

Data warehousing has been at the heart of data analytics solutions for years, effectively managing an organization's most valuable resource: data [1] [2]. But the advent of big data and the ongoing need to experiment with vastly expanded amounts has had an impact on the data warehouse ecosystem [3] although some researchers [4] predicted that data warehouses would be built with big data technologies replacement. Most data warehouse teams support its extensibility to meet the needs of Big Data To accomplish this convergence, the critical components of Big Data—quantity, variety, speed, and accuracy, or 5Vs—must be provided —includes stages of the decision-making process [5]. Traditional data warehouses [6] often fail to meet a wide range of needs, penalize truly for-profit organizations and run counter to the primary objectives of decision support systems [7] External big data (related to their activities); to help managers make informed decisions for organizations using data warehouse technology Enhance your existing data warehouses by introducing platforms Traditional relational database-based data warehousing [8], on the other hand, it is not suited to deal with large external data, characterized by increasing data formats and heterogeneous data formats of internal and external data Effectively coexisting requires distributed, plan- free systems, ecosystems revitalization of this vital system helps. Existing research in literature focuses on converting classical relational data warehouse [9] [10] directly into Column-oriented NoSQL Data Warehouse (CN-DW) [11] to overcome this problem but the advanced work based on clustering algorithms [12] small and has many

flaws. This highlights the question of how to design an optimal CN-DW. We analyze the previous CN-DW models in this study and identify any shortcomings. This study aims to address the current limitations in the design of CN-DW by analyzing and comparing existing models. We will highlight the strengths and weaknesses of each method, with a particular focus on the limitations of previous work that attempted to convert classical relational data warehouses directly into CN-DW. We will also examine advanced efforts based on clustering algorithms, which, although promising, have significant flaws. Through a detailed technical comparison, our objective is to assess the state of the art, identify unresolved issues, and suggest potential improvements as future work for designing an optimal CN-DW.

The structure of the remainder of this paper is outlined as follows: Section 2 delves into related optimized and unoptimized approaches. Subsequently, Section 3 presents a comprehensive comparative study of the existing methodologies. Section 4 concludes our investigation with a discussion and proposes avenues for future research.

2. Related work

Several significant contributions in the existing literature shed light on the development and implementation of data storage techniques [13] [8]. One study [14] delves into methods for transferring data from data lakes [15] to data warehouses, with a specific focus on relational databases. While it doesn't directly tackle large data warehouses in NoSQL databases, it offers valuable insights into data migration pro-

* Corresponding author.

E-mail address: mouhihamohamed@gmail.com (M. Mouhiha).

cesses. Another article [16] investigates the use of NoSQL databases for large-scale data analytics within data warehousing, highlighting the necessity of adapting traditional methods for big data environments. Additionally, an exploration [17] addresses the impact of NoSQL databases on big data warehouses and ETL processes. Further, research [18] explores the use of big data marts employing key-value stores in select NoSQL databases, presenting scalable data warehousing solutions. And introduces Big-Parallel-ETL, which leverages Graph-oriented NoSQL data as an alternative to traditional approaches, addressing big data challenges. Moreover, a comparison study [19] evaluates NoSQL database performance for big data analysis, providing insights into scalability and performance considerations. Lastly, an investigation [20] delves into OLAP cube implementations using NoSQL databases, particularly focusing on Oracle NoSQL, and bridging traditional OLAP techniques with modern data warehouse designs. While these studies collectively contribute to optimizing data warehousing techniques, our research takes a unique angle by focusing on a specific optimization aspect. Unlike prior works that explore general optimizations, our study targets a specialized optimization strategy tailored to meet executive objectives in the big data era. Specifically, we will survey techniques aimed at enhancing decision support systems through the integration of diverse data types within big data warehouses built on NoSQL databases. We will also explore the importance of columnar storage [21] for seamless integration and ongoing support for diverse data types, aligning with the evolving demands of decision support [22] systems in contemporary data environments. Furthermore, our research will delve into two types of approaches in this subject: optimized and unoptimized methodologies, which will be explored in this paper.

2.1. Classical methods

This article [23] analyzed the use of column-oriented NoSQL systems in implementing the multidimensional data warehouses. Tables in the column-oriented NoSQL models have a schema that allows for the number of columns to differ per record. The shared row key and column families of each row organize the physical storage, whereby the vertical data partitioning is implemented. A column family is a collection of columns, each qualified by an atomic value. This paper presents three logical column-based models. The two first models do not split the data resulting in some redundancy since all elements related to a fact and its associated dimensions are stored on one table. The first model (MLC0) saves a data by being grouped in one column family; the second model stores each dimension and fact separately. The third model (MLC2) splits the data into several tables, thus reducing the duplication. To test their approaches, experiments were conducted on a cluster of 3 PCs, each equipped with 4 core-i5 processors, 8 GB of RAM, and 2 TB of storage. The data, generated using an extended version of SSB, was loaded into HBase from CSV files. The benchmark modeled a typical star-schema with a “LineOrder” fact table and four dimensions, using different scale factors ($sf=1$, $sf=10$, $sf=100$). Aggregates were computed with Hive on HBase, applying aggregation functions (max, min, sum, count) at various levels of data aggregation. Their experimental work demonstrates the instantiation of a data warehouse using three distinct approaches, each with its own strengths and weaknesses. The MLC2 model, which minimizes disk space usage due to the absence of dimensional redundancy, is notably less efficient in query processing, often requiring multiple joins. On the other hand, the simpler MLC0 and MLC1 models show no significant differences in performance. The study highlights that converting between these models is straightforward and comparable in time to loading data from scratch. However, one conversion—merging data from multiple tables (MLC2) into a single table—proved to be particularly time-consuming. Additionally, interesting results were observed when computing OLAP cuboid lattices using column-oriented models, which appear to be a reasonable approach within a big data framework. Scalability is a key factor in the effectiveness of big data systems, particularly in real-time environments. The

experiments were conducted on a cluster, demonstrating the models’ scalability and adaptability to large datasets. The results suggest that, depending on the model’s complexity and the system’s goals, scalability can be maintained, albeit with trade-offs between storage efficiency and processing performance. This insight is essential when considering the implementation of similar systems in large-scale, real-time environments, as it provides a foundation for evaluating scalability in various deployment contexts.

This article [24] focuses on transforming a version of the model (multidimensional conceptual) into two NoSQL models that are different from each other. First, it analyzes the ways to convert this model into a column-based one and document oriented. In the column-oriented model, it partitions the facts and dimension attributes. Facts and dimensions are kept in separate columns family/collection. To connect these two organizations (dimension-fact), the dimension identifier is replicated in a column family collection that represents a fact. In the document-oriented model, every multidimensional schema is converted into a large set of documents. Every fact becomes a document, each measure is an elementary attribute, every linked dimension identifier to be simple attributes while everything of all the dimensions become documents. And then the hierarchy transformation into both the column-oriented and document-oriented models is described. In the column-oriented model, each multidimensional schema is mapped into a key space such that one fact corresponds to one column family and so on. In the document-oriented model, every multidimensional schema is translated into a collection of documents where facts are represented as documents; dimension identifier becomes an attribute which identifies only one dimension and hierarchy become an attribute. To evaluate the performance of the implemented NoSQL data warehouses, two key metrics were selected: “Write Request Latency” and “Read Request Latency.” The first metric assesses the system’s speed during data loading, while the second measures the system’s responsiveness to user queries. They chose a set of six queries for this purpose, divided into two categories. The first category involves simple queries that increase the number of dimensions and attributes to test the decision system’s performance when joins are present. The second, more complex category, uses operators and is integrated with the TPC-DS benchmark. The evaluation began by comparing the data loading times (Write Request Latency) of the DWs. It was found that the DW built on Cassandra outperformed the one built on MongoDB, with Cassandra loading 1,000,000 rows in 203.23 seconds compared to MongoDB’s 437.36 seconds. Following this, they tested the systems’ behavior against the query sets (Read Request Latency). Initial results indicate that the document-oriented NoSQL data warehouse is more efficient in handling the two categories of queries, with hierarchical transformations showing the best performance. Loading times varied among the systems, with column-oriented DWs, like Cassandra, having an advantage due to their efficient use of memory space and effective data compression, which is attributed to column redundancy. A notable difference between NoSQL systems is their query performance. For queries requiring multiple attributes, column-oriented approaches like Cassandra might be slower since data is not centralized, which gives MongoDB an edge in such scenarios. They explored the automatic transformation of multidimensional conceptual models into two distinct NoSQL models: column-oriented and document-oriented. Their evaluation demonstrated that column-oriented systems like Cassandra outperformed document-oriented systems like MongoDB in data loading times. However, when processing complex queries involving multiple attributes, MongoDB demonstrated better performance due to its centralized document structure, while Cassandra’s distributed column-based model was slower. This trade-off between data loading efficiency and query performance highlights the scalability differences in NoSQL systems. These insights are particularly relevant in the context of big data, as they demonstrate that different models can be optimized for scalability depending on the specific requirements of the system, such as data volume and query complexity.

The article [25] introduces three methods for implementing large-scale data warehouses using a column-oriented NoSQL DBMS. The first method involves storing facts and dimensions in separate tables, using only simple attributes to represent both measure and dimension attributes. The second method stores both facts and dimensions in a single table, again using simple attributes. The third method also stores facts and dimensions in a single table but employs composite attributes for representation. These methods are outlined below. For the NLA approach, the dimensional model of the data warehouse is mapped using a normalized approach, similar to relational databases. Dimensional models are converted into relational logical models, with facts and dimensions stored in separate tables. To link these entities, the dimension table identifier is duplicated in the fact table. The following rules apply when implementing this approach from the conceptual model:

- (R1) Each fact becomes a table called a fact table (FT), and each dimension becomes a dimension table (DT).
- (R2) Each measure is translated into a simple attribute.
- (R3) Each dimension D and its attributes are mapped into DT as simple attributes, while FT includes a simple attribute (the reference to the related dimension).

The second approach transforms the conceptual data model into a large table structure known as BigFactTable, which includes both facts and joined dimensions. Unlike the normalized approach, this method denormalizes the schema by combining facts and dimensions into a single table. This technique is common in data warehouse modeling, especially for managing dimensional hierarchies. To map measures and dimensions into a logical model, this approach uses simple attributes as supported by columnar NoSQL models. The following rules are followed:

- (R1) Each fact and dimension is converted into a large table called BigFactTable (BFT).
- (R2) Each measure is translated as a simple attribute within BFT.
- (R3) Each attribute of all dimensions D is translated into a simple attribute.

The DLA-CF approach transforms the conceptual data model into a logical column-oriented NoSQL model, similar to the denormalized logical approach. However, this method uses composite attributes to map measures and dimensions instead of simple ones. Each dimension is mapped into a column family, with attributes belonging to the same dimension grouped together. The following rules apply:

- (R1) Each fact and dimension is converted into a table called BigFactTable (BFT) with composite attributes (column families).
- (R2) Each measure is mapped as a simple attribute and included in a column family within BFT.
- (R3) Each dimension D is translated into a composite attribute.

They conducted tests using SSB data generators available in both normalized and denormalized formats, filling the data warehouses NLA-SSB, DLA-SSB, and DLA-CF-SSB with a scale factor of 1000. This setup allowed them to generate a data table with 6×10^9 tuples. For the experiments, they used a set of eight queries divided into two categories. The first category, consisting of four queries, progressively increased the number of dimensions involved when aggregation was performed, with each query using one attribute per dimension. The second category also consisted of four queries but focused on one dimension with a gradual increase in the number of dimension attributes during aggregation. The objective was to study the impact of execution times between normalized and denormalized approaches using queries involving attributes from different dimensions during aggregation. For this, they conducted two tests. In the first test, they applied the first set of queries to the data warehouses NLA-SSB, DLA-SSB, and DLA-CF-SSB. The results showed

that the denormalized data warehouses DLA-SSB and DLA-CF-SSB outperformed the normalized NLA-SSB warehouse, with execution times up to three times faster. In the second test, they observed differences in query execution times across each data warehouse, with DLA-CF-SSB holding an advantage. This test involved queries focusing on a single dimension during aggregation, where NLA-SSB required a join between the fact table and the dimension (Part). Their findings demonstrated that denormalized approaches, such as DLA-SSB and DLA-CF-SSB, significantly outperformed normalized approaches like NLA-SSB in query execution times. This was especially evident when aggregating data across multiple dimensions, where the denormalized models required fewer joins, leading to more efficient data retrieval. These results underscore the importance of selecting the appropriate data model for scalable performance, depending on the size and complexity of the dataset. This study highlights how NoSQL models can scale effectively in large-scale environments, offering insights into their performance trade-offs for real-time big data applications.

This article [26] looks at a variety of configurations for implementing physical data warehouses to make the Star Schema Benchmark work in many NoSQL databases. In particular, it discusses the use of HBase – a column-oriented NoSQL database and different approaches to structuring attributes into CFs. With HBase each CF is stored in a different file. In the case of a query that requires data from two or more CFs, it should read each HFile and join them based on rownum in order to restore the tuple. They indicate that various business situations might be served a lot better by different physical data warehouse structures. They propose three physical data warehouse designs:

1. SameCF: This schema physically locates all the attributes within a single CF.
2. CNSSB: Each dimension is located in the different CFs.
3. FactDate: Some of the most common dimensions are often grouped together with the fact table.

They initially implemented the CNSSB design, which is user-friendly but requires OLAP queries involving any dimension from the data warehouse to process two separate column families (CFs)—one for fact attributes and another for aggregated dimensions. The second design, SameCF, stores all attributes in a single CF to improve the performance of OLAP queries involving a large number of dimensions. However, while this approach generally enhances performance, it can reduce efficiency for OLAP analyses that involve only a small set of attributes and dimensions. Then introduced a third design called FactDate. This approach combines the LineOrder and Date tables into one, offering a middle ground between CNSSB and SameCF. FactDate is designed to enhance the performance of OLAP queries that rely on both the fact table and the Date dimension, given that many analytical queries are time-related. The queries proposed for this design are bidimensional, and it's crucial to vary the query selectivity when creating benchmarks. The OLAP queries in the SSB benchmark use uniformly distributed predicates. Two new queries were introduced: Qnew1 and Qnew2. Qnew2 is a drill-down query that transitions analysis from year to month, moving from the d_year to the d_yearmonthnum. The study evaluated the performance of the three designs—CNSSB, SameCF, and FactDate—against the SSB workload and the proposed queries. It also examined the impact of increasing data volume on performance. Tests were conducted on a 4-node cluster, with each node having a quad-core CPU (3.0 GHz), 16 GB of RAM, a 1 TB SATA disk, and a 1 Gb/s network. One node served as a dispatcher (namenode) while the other three acted as workers (datanodes). The software environment included CentOS 7.0, HBase 0.98.13, Hadoop 2.4.1, and ZooKeeper for data partitioning. The dataset was generated using the SSB's data generator, joined into a single denormalized CSV file, loaded into HBase via HFile conversion, and distributed across datanodes using MapReduce jobs. Each query was implemented in Java (version 1.8) and executed at least five times to calculate the average elapsed time. The results showed that queries performed significantly

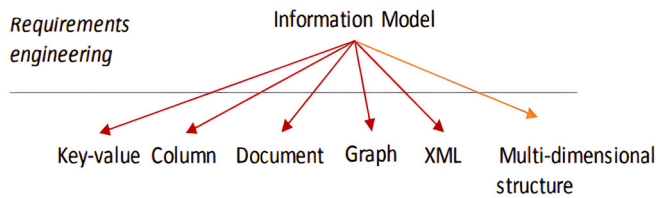


Fig. 1. Information model conversion [27].

worse with SameCF, producing longer processing times compared to the other designs. However, SameCF excelled in scenarios with high-dimensional queries, while FactDate and CNSSB were better suited for low-dimensional queries. When handling single-dimensional queries involving the Date dimension, FactDate provided the best performance. The experiments conducted on a 4-node cluster with 1 TB of storage and 16 GB of RAM per node revealed that each design offered trade-offs in performance depending on the query type and data volume. SameCF, which stores all attributes in a single column family, demonstrated better scalability for high-dimensional queries but struggled with processing times in low-dimensional scenarios. Conversely, FactDate, which combines the fact table with the Date dimension, provided the best performance for time-related, single-dimensional queries. By testing these designs on large-scale datasets generated using the Star Schema Benchmark (SSB), the study demonstrated the importance of tailoring the physical data warehouse structure to the query workload to achieve optimal performance and scalability. These findings highlight the critical role of selecting appropriate NoSQL design strategies to ensure efficient data retrieval and processing in big data applications.

In this paper [27], the authors provide guidelines that can allow for the transfer of information from requirements engineering stage involved with a data warehouse toward the logical model used regarding NoSQL databases as shown in Fig. 1, leading to NOSOLAP. It shows the benefits of NOSOLAP in relation to both ROLAP and also MOLAP. In this article, they provided different details on OLAP models like ROLAP and also Cassandra. Nevertheless, of great interest to us are the proposed rules that transform the information model (the relational model) into Cassandra's logical one. These rules are described below:

- Rule 1: Every single element of the information model transforms into a table in Cassandra. That is, every occurrence of the information model (Fig. 1) provides us with a table.
- Rule 2: Each 'Calculation from' becomes a Cassandra table attribute.
- Rule 3: All the category attributes from the information model become columns of Cassandra table.

Thier article outlines a framework, NOSOLAP, which offers a method for transitioning from traditional data warehouse requirements to a logical NoSQL database model, specifically Cassandra. This framework addresses scalability challenges in OLAP (Online Analytical Processing) systems by proposing a set of rules that guide the transformation of relational data models into Cassandra's column-family architecture. These rules ensure that every element of the relational model becomes a table in Cassandra, and calculation dependencies are translated into table attributes. By following these structured transformations, NOSOLAP leverages Cassandra's distributed architecture to handle large-scale data warehouses efficiently. This scalability is achieved through the decentralization of data, enabling the system to scale horizontally as data volume grows, making it suitable for environments where high availability and distributed querying are required. This approach bridges the gap between ROLAP (Relational OLAP) and NoSQL systems, ensuring that large-scale OLAP workloads can be handled more effectively. They provide valuable insights into handling scalability in column-oriented NoSQL databases by using k-means clustering to optimize the organization of column families. Their work demonstrates that carefully structuring data into fewer column families based on clustering reduces

query execution times significantly in distributed environments, which directly addresses the issue of scalability. Additionally, they emphasize that optimizing the number of clusters and column families is key to maintaining performance at scale, as shown in their experiments with varying cluster sizes. This suggests that adopting similar clustering techniques and continuously adjusting the structure of column families can improve both the scalability and efficiency of data models in large-scale systems.

2.2. Optimized approaches

This article [28] describes a new approach to transforming the relational data warehouse into a column-oriented NoSQL Data Warehouse using kmeans clustering techniques. The issue discussed in this paper is the aspect that column-oriented NoSQL systems no sql data warehouses implementations usually require denormalization techniques. But the current methods do not take into account clustering techniques to form the column families housing data used for query execution. Moreover, these approaches do not monitor the quantity of column families to which multi-dimensional searches are targeted resulting in data warehouse administration complicated and reduction potential benefits from clustering attributes into columns. Therefore, to overcome the mentioned concerns and achieve better weight distribution between the column families when implementing data warehouses on a column-oriented NoSQL databases, a proper design methodology needs. This is how they suggest their own approach. To model the Column NoSQL Data Warehouse shape, they adopt a four-step process detailed in the subsequent subsections, which are depicted in Fig. 2 summarizes the general principle below.

- Pulling all the attributes from the fact table and also dimension tables.
- Grouping the attributes and generating the column families with k-means.
- Creating a logical schema for the CN-DW.
- Data preparation and data loading in the CN-DW.

First, they assume all the attributes in workload queries from Select and Where clauses to build an attribute usage matrices. These matrices show which attributes are used by each and every query. To build column families in the NoSQL databases, they suggest a procedure for clustering the increasingly accessed attributes. This grouping then results in the column families that are a logical schema of CN-DW. They decided to apply the k-means algorithm that divides points into clusters of K in order for a minimum summation of squares arises. For their experiment, they developed the RDW2CNoSQL tool using Java. They tested it with the TPC-DS benchmark, focusing on a sales fact table and nine associated dimension tables. Out of the 99 queries provided by the TPC-DS benchmark, they selected 19 distinct queries that involved accessing all attributes in the sales fact table and its dimensions. The evaluation was conducted in two storage environments. The first was a relational, non-distributed setup using a machine with an Intel Core i7-4790S CPU 3.20 GHz, 8 GB of RAM, and a 500 GB disk, running on a 64-bit Ubuntu 14.04 LTS operating system as a PostgreSQL server. The second environment was a distributed NoSQL system, consisting of a cluster with one master server (NameNode) and three slave machines (DataNodes). The NameNode had an Intel Core i5-3550 CPU 3.30 GHz, 16 GB RAM, and a 1 TB SATA drive, while each DataNode had an Intel Core i5-3550 CPU 3.30 GHz, 16 GB RAM, and 500 GB of disk space. These machines also ran on 64-bit Ubuntu 14.04 LTS with Java JDK 8. For the distributed environment, they used Hadoop (v2.6.0) and MapReduce for processing, with HBase (v0.98.8) for storage. ZooKeeper was employed to monitor the status of distributed data in the RegionServers, and Phoenix (v4.6.0) along with Squirrel SQL Client were used to enhance data manipulation and boost HBase performance. To streamline the transfer of PostgreSQL data to HBase, they incorporated Sqoop into

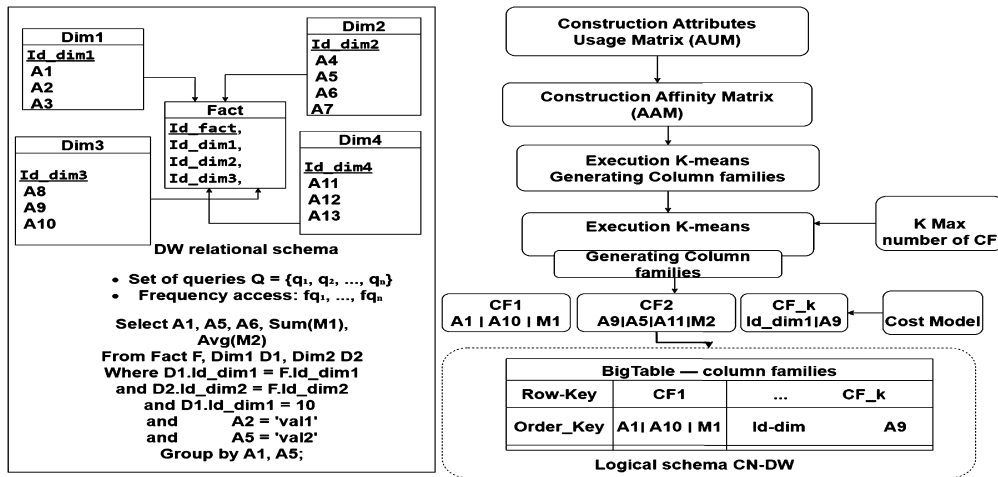


Fig. 2. k-means method for designing a columnar NoSQL data warehouse [28].

the RDW2CNoSQL tool. They compared three distinct approaches, including two existing methods. In the first method, the fact table and its 9 dimension tables are stored in one HBase table with only one column family for all attributes, known as the Flat schema. In the second method, the fact table and its 9 dimension tables are stored in one HBase table with 10 column families, one for each table, known as the Naïve schema. These were compared to their suggested approach for implementing CN-DW into the HBase DBMS (all data are stored in one HBase table). When analyzing the impact of the number of column families on query execution time, the results show that query runtime is influenced by the modeling of column families. Specifically, the k-means method with $k = 11$ reduced global query execution time by up to 19.20% compared to the naïve approach and by 34.38% compared to the flat approach. This improvement occurs because the k-means method calls fewer column families, while other methods call a larger number of column families, increasing the cost of combinations and the reconstruction of intermediate results. To optimize query runtime in an HBase-based data warehouse, consider the following: (1) limit the number of columns per family, and (2) select an optimal number of column families. Testing with different cluster sizes ($k = 6, 7, 8, 9, 10, 11, 12, 13$) indicates that creating too many column families is not advisable. It is advisable to test with different numbers of clusters and then choose the number of clusters that gives the best execution time. It is not advisable to rely solely on the elbow method or silhouette score to choose the best k number of clusters. This approach demonstrates how clustering techniques, such as k-means, can be employed to optimize column family design in column-oriented NoSQL data warehouses. Their experiment, which compares the scalability of various schema models in a distributed NoSQL environment, highlights the performance benefits of reducing query execution time by limiting the number of column families. Specifically, their method achieves significant time reductions by clustering attributes optimally, which directly impacts the system's ability to handle larger datasets and more complex queries in real-time. This demonstrates that well-designed clustering can improve the scalability of NoSQL data warehouses, making it applicable to large-scale big data scenarios where efficient query processing is essential. However, further exploration into how these methods perform under varying data loads and real-time constraints would provide additional insights into their scalability in different big data environments.

This paper [29] proposes a new technique which uses clustering methods to generate data warehouse column family schema that could be used in an analysis process when developing the distributed warehousing with NoSQL Database using a Column Family. In the schema design stages for a column family, the problem is selecting between a relational conceptual model and also relational logical model. Therefore, although NoSQL modeling is taken into the account it still contains

various relational Data Warehouse models and also the implementation processes for data warehouses built upon such systems often use de-normalized techniques. However, the researchers tried to develop new data models (as demonstrated in Fig. 3) that would allow for a faster response time for the processing of such complex queries. They studied the effect of each column on family-based parameter on data warehouse performance. In response, they suggested a clustering-based design for building column family schemas to improve the efficiency of query performance gains. Their main objective is to reduce the absolute amount of data examined in an OLAP query as much minimally as possible through vertical pre-partitioning on column families, which determines how many columns are contained within these groups before practical implementation. In this way, they strive to lessen the effect of workload fluctuation on the adopted data distribution strategy. To elaborate on their methods, there are two primary stages. The first method involves forming the query groups where they extract the Attribute-Query Matrix (AQM) and then from it get to Extract Query Distance Matrix followed by clustering of queries using k-medoids. The second major step is building the query column family schema, which they implemented by the use of a Particle Swarm Optimization (PSO) metaheuristic algorithm. Their approach was evaluated by using the TPC-DS benchmark, which included the STORE_SALES fact table along with 9 dimension tables. For the query workload, they chose 19 different queries with an access to 67 attributes through use of all schema elements from STORE_SALES facts and their dimensions tables using joins in a logical rather than system level. For the experimental setup, they established two storage environments: the first is a centralized relational environment, using PostgreSQL as an exclusive server to contain the relational data warehouses. The second is a distributed NoSQL storage system. Also, they implemented the TPC-DS using two already existing techniques together with their own suggested method in HBase. In the first method, the STORE_SALES fact table and its 9 dimension tables are stored in an HBase flat schema – a single column family for all attributes. In the second method, they applied a table with 9 column families – one for each dimension table (called Naïve schema). The final approach involved CN-DW as described in their method, with three different schemas named schema A, schema B and also schema C. In the first two (schema a,b) all queries are grouped into a single group $G1(k = 1)$, while on the second one $k = 3$. The total execution time for queries (q4 to q19) is discovered as 949.66 seconds. Unlike the first time, the longer procedure of difficulty learning related to schema B ($k = 1$), Naïve schema and flat shape upsurge which exceeds for totally besides 1 case in number equalizer where willingness really are reached by approx. Their experiments, conducted using the TPC-DS benchmark, demonstrated that distributed NoSQL systems such as HBase can handle large volumes of data with improved scalability when compared to traditional relational systems.

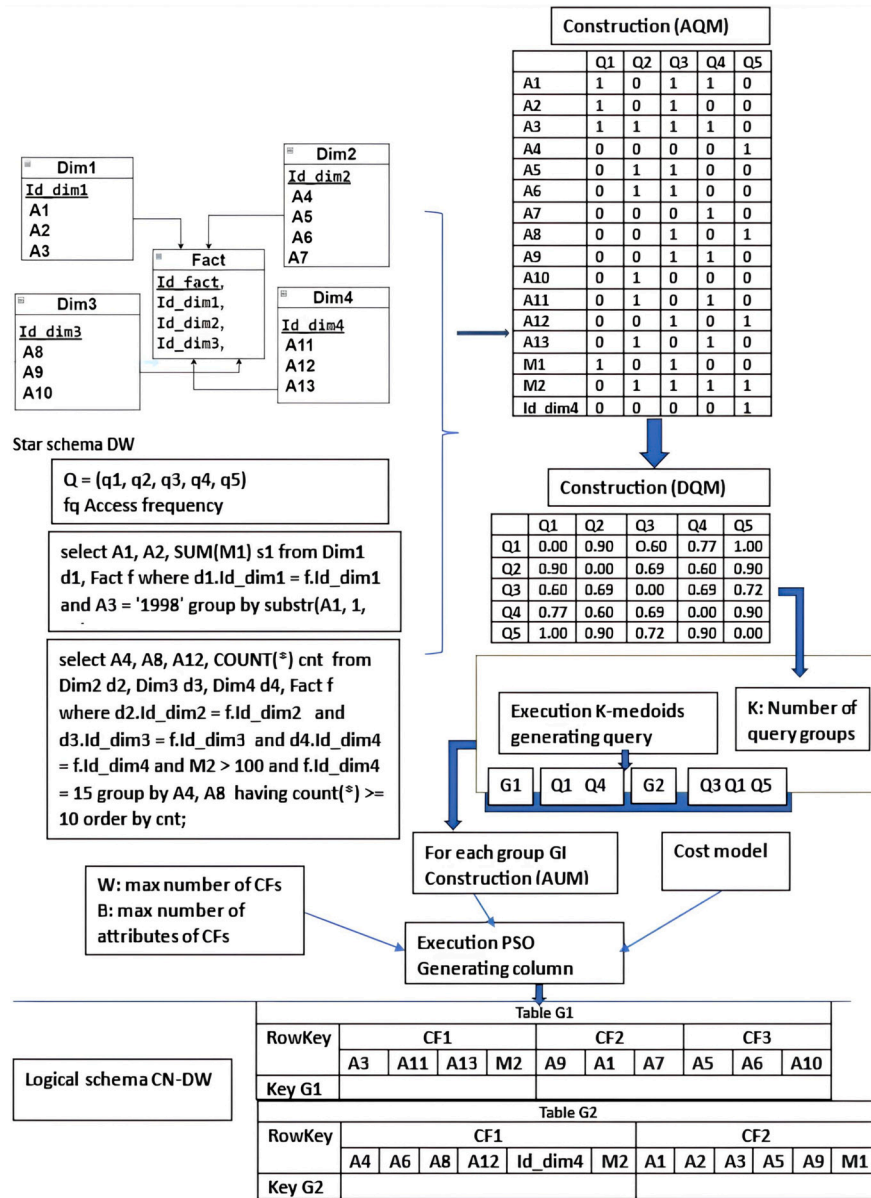


Fig. 3. Clustering-based method for designing a NoSQL data warehouse [29].

This evidence supports the scalability of the methods discussed in this manuscript, particularly in handling fluctuating workloads in big data environments. Furthermore, the partitioning strategy described ensures that query performance remains efficient even as the dataset size grows, making the proposed methods suitable for real-time, large-scale implementations.

In this paper [30], the authors propose a method for implementing a relational data warehouse (star chart or snow chart) using a columnar NoSQL logic model. This approach is presented as a practice partial function. It relies heavily on statistical analysis of questionnaires and uses both qualitative and quantitative data processing information.

1. Extract elements from fact tables and assumptions. In this step, all the attributes associated with the query load (contained in the Select and Where components) are processed, and the authors used the principle of attribute affinity to construct feature sets.
2. Create a family of columns. The goal is to define a NoSQL logical schema for a repository that optimizes data access queries. The authors propose a solution that involves aggregating frequently asked questions. Two different algorithms were chosen to implement this:

(1) the metaheuristic OEP and (2) the k-means algorithm. Their choice is due to the need to control the number of classes (column families) in k-means, which is useful because in addition to wanting to limit the number of column families, OEP allows to consume the number of objects as it is in each group (column family) and carries the same value as k-means. This helps create color families with the same number of elements to balance the load.

- The Optimization by Particle Swarm (OEP) algorithm draws inspiration from the swarm behavior of insects and foraging animals. Initially, these clusters are randomly distributed in space, and each particle has a random velocity. The particles then move to the required location based on a few pieces of information. In other words, each particle must determine its next motion and new velocity. To achieve this, we horizontally combine three parameters: (1) current speed, (2) optimal performance: each particle can measure its position and optimal performance for each by recalling previous success points, and (3) best neighbor performance (informant): each particle based on best neighbor performance Can query its immediate neighbor to determine which

one to move. Particle swarm algorithms can be applied to discrete and continuous data.

- The use of k-means for feature clustering requires different points and an integer k as input. The objective is to divide the points into k groups, minimizing the sum of the squares between the points in the group and its center. In this case, the points represent attributes, and their distances represent relationships. k-means takes the attribute affinity matrix (MAA) and the number of clusters k as input, and returns column families.

They chose four construction methods for column families:

1. All attributes in fact and dimension tables are placed in the same column family (flat approach).
2. Fact and dimension tables each correspond to a column family (naive approach).
3. Families are determined based on groupings obtained using the OEP algorithm.
4. Column families are obtained using the k-Means algorithm.

They developed a tool named TA-EDRC (Automatic Transformation of a Relational Data Warehouse to Column-Oriented NoSQL) using Java, which implements the OEP-FC and k-Means methods. For the data warehouse, they used the TPC-DS benchmark implemented under PostgreSQL, which models various aspects of commercial operations across three sales channels: in-store, catalog, and online. TPC-DS uses a constellation schema with 17 dimension tables and 7 fact tables. In this case, they used the store_sales fact table and 9 dimension tables. The TPC-DS data generator (DSDGEN) created data files (in .dat format) with a scale factor (SF) of 100, resulting in 100 GB of data for queries on a 19-node cluster. For query workload, the TPC-DS benchmark offers 99 queries. They selected 19 distinct queries that use the store_sales fact table and its dimensions. These queries were organized based on the number of tables (fact and dimension) accessed and the number of selection predicates defined on various attributes of the tables. The queries were classified into six categories: SQ1 to SQ6. The experimental setup involved two storage environments. The first was a non-distributed relational environment using an Intel-Core i7-4790S CPU@3.20 GHz with 8 GB RAM and a 500 GB disk, running Ubuntu-14.04 LTS 64-bit and serving as the PostgreSQL server for relational data storage. The second was a distributed NoSQL environment consisting of a cluster with a master server (NameNode) and 19 slave machines (DataNodes). The NameNode had an Intel-Core i5-3550 CPU@3.30 GHzx4, 16 GB RAM, and a 1 TB SATA disk, while the DataNodes had Intel-Core i5-2400M CPUs, 8 GB RAM, and 300 GB disk space, all running Ubuntu-14.04 LTS 64-bit with Java JDK 8. They used Hadoop 2.6.0 and HBase-0.98.8-Hadoop2 for managing data in the distributed environment. To enhance HBase's performance and simplify data handling, they added a SQL layer called Phoenix (v4.6.0) and used a JDBC client named SquirrelL for querying HBase data. The NameNode also served as the master server for HDFS and HBase's Zookeeper, while the DataNodes acted as HBase Region Servers. They observed that the execution time of Rq3 varied between different warehouse schemas. The OEP-FC method yielded better results, with an execution time of 19,140 seconds for $W = 10$. This query involved attributes from three column families with balanced, small data sizes. However, with the k-Means algorithm, execution time increased to between 50,438 and 53,287 seconds for $5 \leq k \leq 7$. This increase was due to the higher number of combinations of operations (selection, filtering, and aggregation) required to determine selection predicates and identify attribute values, leading to more memory-intensive data processing. In these warehouse schemas, the query accessed four non-homogeneous column families with data sizes that were either too large or too small, requiring HBase to retrieve data and intermediate results from numerous HFile data blocks spread across multiple nodes. This distribution hindered the flow of intermediate results, resulting in longer execution times. On the other hand, for k-Means with $9 \leq k \leq 12$, execution times

were closer, ranging from 32,782 to 38,912 seconds. These schemas involved 3 to 5 column families with medium data sizes, reducing memory overload and resulting in shorter execution times compared to $5 \leq k \leq 7$. Conversely, the "Naïve" method required only 30,629 seconds, as it involved fewer operation combinations across two column families with different data sizes (one with 21 large attributes and the other with 12 medium-sized attributes), offering a balance between data and processing. This balance reduced query execution time. The results also showed the interdependence between the number of column families accessed by the query and their data sizes. The OEP-FC method performed best when $8 \leq W \leq 11$, likely due to its consideration of column family size when defining the maximum number of attributes per family to avoid penalizing query processing time. Experiments using the TPC-DS benchmark on both centralized and distributed environments demonstrated significant scalability improvements, with the OEP-FC method achieving the best results in balancing query workloads across multiple column families. These techniques are particularly well-suited for large-scale implementations, where fluctuating workloads and complex queries are common. By addressing data distribution and processing efficiency, the proposed approaches ensure that the system can handle increasing data sizes and workloads without sacrificing query performance.

3. Comparative study

To create multidimensional data warehouses, the column-oriented paradigm is used as an alternative to the relational approach [28]. It is the first remedy used in this situation. HBase and Cassandra DBMS are commonly used in column-oriented warehousing. HBase can manage massive amounts of data in a distributed setting. It is built on top of Hadoop (it saves data in the HDFS storage area).

3.1. General comparison: advantages and disadvantages

To evaluate the techniques, we will examine the goal, benefits, and drawbacks of each, as detailed in the Table 1.

3.2. Technical comparison

The stages involved in changing the conceptual model into a column-oriented or document-oriented NoSQL model are summarized in the Table 2. The following characteristics are taken into account:

1. A multidimensional type diagram might be a constellation, a flake, or a star.
2. The type of NoSQL logic model employed. The model can be either column- or document-oriented.
3. The NoSQL technology used to construct the data warehouse (HBase, Cassandra, MongoDB, etc.).
4. Conversion type: direct or optimized
5. Used technique: k-means or K-medoids.
6. The type of storage classification employed in the data warehouse's implementation. These kinds are divided into two categories: normalized schema and denormalized schema:
 - **Denormalized approach:** the goal of these efforts is to provide a storage scheme that combines fact and dimension tables into a single table. Three approaches are frequently used to construct a column family diagram:
 - The first logical data model:** all attributes from fact and dimension tables are combined into a single column family.
 - The second logical data model:** a column family for each dimension table.
 - The third logical data model:** a family of columns for each dimension table and a family of columns dedicated to the facts table. However, each column family is referenced by a unique RowKey.
 - **Normalized approach:** utilizes various tables (independent tables) to physically store fact and dimension tables. Each dimension

Table 1
Advantages and Disadvantages of Approaches.

Approach	Goal	Advantages	Disadvantages
[30]	The implementation of a relational warehouse in the form of a semi-automatic process, based on a statistical analysis of the most frequent queries.	- The response to queries is quick and efficient, yielding better results than K-means.	- The HBase system uses a significant number of HFile data blocks, which penalizes the flow of intermediate results and consequently requires a high execution time.
[23]	New guidelines for transforming a multidimensional conceptual model into NoSQL models column/document oriented	- Looked into document-oriented NoSQL models - Direct conversion - Outperformed column-oriented by document-oriented - Snowflake pattern considered	- Didn't look at NoSQL systems in terms of more advanced queries (roll-up, drill-down) - Inefficient in dealing with more complicated queries
[28]	The relational data warehouse is transformed into a columnar NoSQL data warehouse using k-means clustering algorithms	- Approach which has been optimized. - Approach that is effective. - Enhanced query performance.	The method involves converting a relational data warehouse into a heavily loaded single-table NoSQL data warehouse. It does not account for properties unused in queries and requires specifying the number of clusters without allowing adjustments to the initial configuration.
[25]	Three approaches for constructing massive data warehouses using column-oriented NoSQL DBMS	- Contrasted two approaches: standardized and denormalized	- Traditional and simple, not optimized
[27]	Guidelines for migrating information from the data warehouse's requirements engineering to the logical model of NoSQL databases (NOSOLAP)	- Demonstrated benefits of NOSQL OLAP	- Not optimal - Ineffective for complex queries
[26]	Studies three physical data warehouse designs to adapt the Star Schema Benchmark for use in NoSQL databases	- Compared three approaches	- Ineffective for dealing with difficult questions - Straightforward transition
[29]	Proposes a new method based on clustering techniques to create a column family schema for building a distributed data warehouse using a column family NoSQL Database	- Approach that is effective. - A novel technique to organizing column families. - Response time to huge requests is really fast.	- The method does not consider properties that are not used in queries. - The strategy necessitates the declaration of the number of clusters while making no improvements to this initial configuration.

Table 2
Technical comparison of existing approaches.

Approach	Classic DW Conceptual Scheme Type	Big DW Type	Big DW Nosql DB Type	Type of Transformation Approach	The Clustering Algorithm	Approach Classification
[23]	snowflake	Column	Cassandra, HBase	Not Optimized	Simple Transformation	Denormalized, Normalized
[24]	Star/ snowflake	Column/ document	HBase	Not Optimized	Simple Transformation	Denormalized
[25]	Star/ snowflake	Column	HBase	Not Optimized	Simple Transformation	Denormalized, Normalized
[27]	Star	Column	Cassandra	Not Optimized	Simple Transformation	Denormalized, Normalized
[26]	Star	Column	HBase	Not Optimized	Simple Transformation	Denormalized, Normalized
[28]	Star	Column	HBase	Optimized	K-means	Denormalized
[29]	Star	Column	HBase	Optimized	K-medoids	Normalized
[30]	Star/ snowflake	Column	HBase	Optimized	OEPCF, k-means	Denormalized

sion table is saved in a table with a column family, while the fact table is stored in a table with a column family.

The choice between normalization and denormalization depends on several key factors in the database design. Normalization aims to reduce data redundancy and ensure data integrity and accuracy. However, this can make data retrieval difficult, especially for complex queries with many tables. On the other hand, denormalization typically simplifies query operations by combining related data into a single table. This

can increase read performance but comes at the cost of potential data redundancy and increased complexity of updates to maintain accuracy.

In data modeling, it is important to consider the nature of read and write operations, the expected size of the database, and the system's performance constraints. While some systems may conceptually use hybrid approaches that combine elements of normalization and denormalization to leverage the advantages of both methods, it is crucial to recognize that this approach remains largely theoretical, with limited academic documentation or structured implementation in real-world systems. The

idea is to use normalization to ensure data integrity by minimizing redundancies and update anomalies, while selectively applying denormalization to parts of the database where query performance is paramount, allowing for faster access to frequently requested data. However, implementing such a hybrid approach requires careful management to balance the benefits and mitigate the drawbacks of each method. Excessive normalization can complicate queries, making them more costly, while poorly managed denormalization can lead to data inconsistencies. Therefore, the choice between normalization and denormalization, or a potential hybrid approach, must be made based on the specific needs of the project and business priorities. In an e-commerce platform, a hybrid approach might be used where product and customer details are stored in a clean, organized way (normalized) to avoid errors. At the same time, the system could keep copies of frequently searched items (denormalized) to allow customers to see them faster. On social media, your profile details might be stored accurately (normalized), while your news feed could be quickly built from a mix of data (denormalized) to show posts, likes, and comments instantly. In banking, your account and transaction data might be kept organized (normalized) to prevent mistakes, but a quick summary of your recent transactions could be stored separately (denormalized) for instant access. This potential balance would make these systems both fast and reliable. Further research and practical experiments are needed to validate the effectiveness of hybrid systems in maintaining data integrity, accuracy, and simplifying query operations.

4. Discussion and future work

The performance of data storage and manipulation methods in column-oriented NoSQL systems varies significantly based on usage scenarios and data structures. On [23] found that the MLC2 model, which reduces redundancy by splitting data into multiple tables, uses less memory compared to the MLC0 and MLC1 models. The findings on [24] showed that column-oriented systems like Cassandra have better data compression and memory usage, while document-oriented systems like MongoDB offer different loading times and flexibility for dynamic queries. The theory is discussed in [25] compared three storage methods and found that denormalized data warehouses are up to three times faster in execution speed, highlighting the benefits of denormalization for performance. [26] analyzed different physical configurations of data warehouses and found that the SameCF design is optimal for queries involving large dimensions, while FactDate performs better for single-dimensional temporal queries. In [28] introduced a method using k-means clustering to transform a relational data warehouse into a column-oriented NoSQL data warehouse, improving query performance by better distributing loads among column families. Their tool RDW2CNoSQL showed slightly better query execution times with a k=11 schema compared to other approaches. In another study, [29] proposed using clustering and particle swarm optimization (PSO) to design column family schemas, which reduced the amount of data examined in OLAP queries and improved response times.

To provide practical insights into these challenges and the effectiveness of various approaches, we examine real-world case studies that highlight the application and performance of column-oriented NoSQL systems in different scenarios. Even though NoSQL is still relatively new, it has been widely adopted in various industries. Many leading Web 2.0 companies, like Google and Facebook, are using NoSQL, but it is also being implemented by organizations in media, online entertainment, and government sectors. Facebook, for instance, introduced a new messaging system built on HBase in November 2010. While MySQL and Cassandra were considered, HBase was selected for its scalability, reliable replication system, and Facebook's prior experience with the underlying technology [31]. Initially, MySQL supported this application, but the system's data volume (25TB per month) required a solution that could handle a high write throughput [32]. Similarly, Nokia transitioned from a relational database to an in-house NoSQL system [33]. Nokia's Places application, which serves as a POI registry for GPS, was originally based

on a MySQL database with about 600 GB of data [33]. However, with a potential merger with Bing Maps and the desire to avoid investing in an RDBMS server upgrade, Nokia opted for a NoSQL solution. This choice allowed for on-demand scaling, a critical feature considering the potential merger, and provided the flexibility needed for the application's evolving schema. The Danish Department of Health uses the key-value store Riak for its medicine card system, which manages drug administration for cardholders [34]. This system was previously based on MySQL but was switched to NoSQL for several reasons: High availability, scalability, and operational improvements. Riak's distributed architecture across multiple data centers met these needs, highlighting that NoSQL isn't just for Web 2.0 companies but is also being used in critical enterprise applications. Google's Bigtable is a distributed storage system designed to handle large amounts of structured data, integral to several of Google's internal services, such as Google Maps, Google Earth, and Google Search. Bigtable's strength lies in its ability to scale horizontally, providing low-latency access to petabytes of data across thousands of servers [35]. Netflix, a leader in online entertainment, uses Apache Cassandra to manage time-series data for its streaming services. Cassandra was chosen for its ability to handle large volumes of write operations, ensuring smooth streaming experiences for millions of users [36]. Yahoo! has been one of the early adopters of Apache Hadoop and HBase for managing its vast amount of user-generated data. HBase, built on top of Hadoop, allows Yahoo! to store and process large-scale data in a distributed environment, supporting its advertising and search services [37]. LinkedIn developed Espresso, a distributed document-oriented database, to handle the vast amount of data generated by its professional networking platform [38]. Finally, Twitter uses Apache FlockDB, a graph database, to manage relationships and connections between users on its platform, underscoring the versatility of NoSQL databases in handling specialized use cases [39].

The clustering and optimization approach outlined in this paper for column-family schema design can be extended to other NoSQL database types such as document-based, key-value, and graph-oriented systems. For document-based systems like MongoDB, clustering can be employed to group similar documents and partition them effectively across shards. This improves query performance, especially for large-scale data with dynamic schemas, as demonstrated by [24], where MongoDB's document-oriented nature facilitated flexible data access and query optimization through Cassandra clustering strategies. Graph databases, like Twitter's FlockDB, also stand to benefit from this approach. Although primarily focused on relationships between nodes, clustering techniques could help optimize the storage and retrieval of graph data by grouping related nodes together, as seen in the application of graph clustering algorithms to improve query times in social network analyses [39]. Clustering in key-value stores may benefit from distributed clustering algorithms like DBSCAN or K-means, which could be adapted to work within environments such as Hadoop's MapReduce. These environments, often employed for large-scale key-value stores, may allow for efficient parallel processing of unstructured data. The key challenge lies in how we may effectively map the unstructured key-value pairs to a feature space where clustering can be applied. This may involve developing transformations to group similar keys (e.g., based on value distributions or semantic similarities), or using dimensionality reduction techniques to aggregate key-value pairs into meaningful vectors suitable for clustering. By applying these transformations, clustering techniques may be extended to optimize and derive insights from key-value databases, similar to how they are used in columnar models. These examples demonstrate that the optimization strategies discussed in this paper are not limited to column-oriented systems but can be effectively applied across a range of NoSQL systems to enhance scalability, performance, and data management.

Overall, these studies suggest that data modeling processes, including denormalization, fragmentation, data compression, and the use of clustering and optimization techniques, are crucial for maximizing the efficiency and performance of OLAP queries in column-oriented NoSQL

systems. Although these optimized studies offer certain advantages, they have two major limitations: the clustering algorithms used, k-means and k-medoids, require specifying the number of clusters without providing an automatic improvement for this initial configuration, and they only consider the attributes used in decision-making queries for designing the distributed data warehouse. This makes the designed CN-DW incapable of meeting new requirements that involve excluded attributes. This comparative study shows that, despite the many solutions proposed in the literature, the challenge of designing an optimized CN-DW remains. In our future research, we will propose the following solutions to address these gaps:

- A new approach for the optimized Columnar NoSQL Data Warehouse. Our focus will be on finding the optimal number of clusters using the most appropriate clustering scheme. In addition, our framework will consider all features of the Relational Data Warehouse (Rel-DW) for comprehensive modeling and targeting. This strategy aims to address the shortcomings identified, ensuring that it is better aligned with the evolving needs of industry leaders.
- Use a hybrid approach that combines normalization and denormalization techniques.
- Performance Optimization of Hybrid Models: Investigate methods to fine-tune the balance between normalized and denormalized data, optimizing performance without sacrificing data integrity. This could involve identifying the optimal scenarios for using each technique, refining algorithms to dynamically switch between normalized and denormalized structures, and quantifying performance gains across different industries.
- Explore the integration of machine learning techniques for adaptive clustering based on developed data models.
- To develop strategies for dynamic adaptation of CN-DW to adapt to changing business requirements and data models over time.
- Measure real-time updates and changes to warehouse layouts based on incoming data.
- Address scalability challenges by exploring distributed computing systems and architectures for CN-DW.
- Monitor and optimize system performance, consider big data and complex queries.

This future research direction aims to contribute to the development and continuous improvement of Columnar NoSQL Data Warehouses by addressing various aspects.

CRedit authorship contribution statement

Mohamed Mouhiha: Writing – review & editing, Writing – original draft, Visualization, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Abdelfettah Mabrouk:** Visualization, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We appreciate and would like to thank the anonymous referees for their constructive comments and suggestions to improve the presentation of this work which was supported by ELITES Laboratory, Chouaib Doukkali University, El Jadida, Morocco.

Data availability

No data was used for the research described in the article.

References

- [1] M. Paliwal, P. Saraswat, Approaches of data warehousing and their applications: a review, *International Journal of Innovative Research in Computer Science & Technology* 10 (1) (2022) 117–121.
- [2] R. Thareja, *Data Warehousing*, Oxford University Press, 2009.
- [3] S.-M. Huang, Y.-C. Hung, I. Kwan, Y.-M. Hung, Developing an active data warehouse system, *Heterogeneous Information Exchange and Organizational Hubs* (2002) 105–122.
- [4] M. Barkhordari, M. Niamanesh, Atrak: a mapreduce-based data warehouse for big data, *Journal of Supercomputing* 73 (10) (2017) 4596–4610.
- [5] D. Kunda, H. Phiri, A comparative study of nosql and relational database, *Zambia ICT Journal* 1 (1) (2017) 1–4.
- [6] D.K. Pande, *Data warehouse techniques in traditional knowledge systems*, 2005.
- [7] R.T. Ng, J. Han, Clarans: a method for clustering objects for spatial data mining, *IEEE Transactions on Knowledge and Data Engineering* 14 (5) (2002) 1003–1016.
- [8] Z. Senwei, Relational database-based method and system for forming hive data warehouse, 2016.
- [9] A. Dhaouadi, K. Bousselmi, M.M. Gammoudi, S. Monnet, S. Hammoudi, Data warehousing process modeling from classical approaches to new trends: main features and comparisons, *Data* 7 (8) (2022) 113.
- [10] A. Yusuf, A design comparison: data warehouse schema versus conventional relational database schema, in: *OcRI*, 2016, pp. 217–222.
- [11] M.F. Al-Ghifari, F.N. Azizah, Development of application for entity-relationship diagram conversion to logical schema of nosql column oriented, in: *2022 International Conference on Data and Software Engineering (ICoDSE)*, IEEE, 2022, pp. 30–35.
- [12] G. Feng, J. Lin, K. Wang, Researches advanced in clustering algorithms, *Highlights in Science, Engineering and Technology* 16 (2022) 168–177.
- [13] M.H.P. Swari, F.A. Akbar, M. Martoni, Data warehouse implementation techniques in data processing (case study data sales at pt spirit sejahtera bersama): data warehouse implementation techniques in data processing (case study data sales at pt spirit sejahtera bersama), *Jurnal Mantik* 3 (3) (2019) 135–142.
- [14] F. Abdelhedi, R. Jemmali, G. Zurfluh, Relational databases ingestion into a nosql data warehouse, *arXiv preprint, arXiv:2203.06949*, 2022.
- [15] P.P. Khine, Z.S. Wang, Data Lake: a New Ideology in Big Data Era, *ITM Web of Conferences*, vol. 17, EDP Sciences, 2018, p. 03025.
- [16] B.F.P. de Oliveira, M. de Carvalho Victorino, M. Holanda, Data warehouse based on nosql: a literature mapping, in: *2021 16th Iberian Conference on Information Systems and Technologies (CISTI)*, IEEE, 2021, pp. 1–6.
- [17] S. Bouaziz, A. Nabli, F. Gargouri, Nosql big data warehouse: review and comparison, in: *International Conference on Intelligent Systems Design and Applications*, Springer, 2020, pp. 392–401.
- [18] A. Khalil, M. Belaissaoui, New approach for implementing big datamart using nosql key-value stores, in: *2020 5th International Conference on Cloud Computing and Artificial Intelligence: Technologies and Applications (CloudTech)*, IEEE, 2020, pp. 1–6.
- [19] A. Thakare, O.W. Tembhurne, A.R. Thakare, S.N. Reddy, Nosql databases: modern data systems for big data analytics-features, categorization and comparison, *International Journal of Electrical and Computer Engineering Systems* 14 (2) (2023) 207–216.
- [20] A. Khalil, M. Belaissaoui, Implementing big olap cubes using a nosql-based approach: cube models and aggregation operators, in: *Enabling Machine Learning Applications in Data Science: Proceedings of Arab Conference for Emerging Technologies 2020*, Springer, 2021, pp. 179–191.
- [21] C. Fang, S. Song, H. Guan, X. Huang, C. Wang, J. Wang, Grouping time series for efficient columnar storage, *Proceedings of the ACM on Management of Data* 1 (1) (2023) 1–26.
- [22] M. Ghaffarzadeh, M. Behartash, M. Azimi, S. Eyvazzade, Evolution and development of decision support systems, 2014.
- [23] M. Chevalier, M.E. Malki, A. Kopliku, O. Teste, R. Tournier, Implementation of multi-dimensional databases in column-oriented nosql systems, in: *Advances in Databases and Information Systems: 19th East European Conference, Proceedings, ADBIS 2015, Poitiers, France, September 8–11, 2015*, vol. 19, Springer, 2015, pp. 79–91.
- [24] R. Yangui, A. Nabli, F. Gargouri, Automatic transformation of data warehouse schema to nosql data base: comparative study, *Procedia Computer Science* 96 (2016) 255–264.
- [25] K. Dehdouh, O. Boussaid, F. Bentayeb, Big data warehouse: building columnar nosql olap cubes, *International Journal of Decision Support System Technology (IJDSST)* 12 (1) (2020) 1–24.
- [26] O.Q.P. over HBase, Physical data warehouse design on nosql databases, in: *ICEIS 2016*, 2016, p. 111.
- [27] D. Prakash, Nosqlap: moving from data warehouse requirements to nosql databases, in: *ENASE*, 2019, pp. 452–458.
- [28] M. Boussahoua, O. Boussaid, F. Bentayeb, Logical schema for data warehouse on column-oriented nosql databases, in: *Database and Expert Systems Applications: 28th International Conference, Proceedings, Part II, DEXA 2017, Lyon, France, August 28–31, 2017*, vol. 28, Springer, 2017, pp. 247–256.
- [29] M. Boussahoua, F. Bentayeb, O. Boussaid, N. Kabachi, A data partitioning optimization approach for distributed data warehouses on column family nosql systems, in:

- Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA), the Steering Committee of the World Congress in Computer Science, Computer ..., 2018, pp. 54–60.
- [30] M. Boussahoua, O. Boussaid, F. Bentayeb, Optimisation des performances dans les entrepôts de données nosql orientés colonnes, in: 17ème Journées Franco-phones Extraction et Gestion des Connaissances, EGC 2017, Hermann Éditions, 2017, pp. 69–80.
 - [31] DBPeditas.com, Facebook's messaging system architecture, <http://dbpeditas.com/facebook>. (Accessed 11 August 2024).
 - [32] K. Muthukkaruppan, Case study: Facebook's messaging system, in: Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011.
 - [33] S. Farrell, Nosql: database choices in cloud computing, Network Computing 15 (3) (2011) 1–9.
 - [34] P. Thorub, Implementing high availability with riak: a case study from the Danish department of health, ACM Queue (2011).
 - [35] S.G. Fay Chang, Jeffrey Dean, et al., Bigtable: a distributed storage system for structured data, ACM Transactions on Computer Systems (TOCS) 26 (2) (2008) 4.
 - [36] P.M. Avinash Lakshman, Cassandra: a decentralized structured storage system, ACM SIGOPS Operating Systems Review 44 (2) (2010) 35–40.
 - [37] D. Borthakur, The hadoop distributed file system: architecture and design, <https://hadoop.apache.org>, 2007. (Accessed 11 August 2024).
 - [38] W.X. Yuan Qiao, Shan Yao, et al., Espresso: a nosql database for linkedin's real-time applications, <https://engineering.linkedin.com/espresso>, 2013. (Accessed 11 August 2024).
 - [39] Twitter Engineering, Flockdb: Twitter's distributed graph database, <https://engineering.twitter.com>. (Accessed 11 August 2024).